

Q1: Scenario: An explorer robot wants to discover all rooms reachable from its current room through connecting corridors. A graph and BFS function signature are provided. Complete the BFS logic inside the function: use a queue, mark visited nodes, and print each node as it is visited. Then take a start node as input and print the final discovery order.

```
#Q1
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'G'],
    'F': ['C'],
    'G': ['E']
}

def bfs(graph, start):
    visited = []
    queue = deque([start])
    seen = set([start])

    while queue:
        node = queue.popleft()
        visited.append(node)
        print(f'Visiting: {node}')

        for neighbor in graph[node]:
            if neighbor not in seen:
                seen.add(neighbor)
                queue.append(neighbor)

    return visited

start_node = input('Enter start node: ').strip()

if start_node not in graph:
    print(f'Node not found. Valid nodes: {list(graph.keys())}')
else:
    result = bfs(graph, start_node)
    print('Discovery order:', result)
```

```
Enter start node: G
Visiting: G
Visiting: E
Visiting: B
Visiting: A
Visiting: D
Visiting: C
Visiting: F
Discovery order: ['G', 'E', 'B', 'A', 'D', 'C', 'F']
```

Q2: Scenario: The warehouse has a 5x5 floor plan where some cells are blocked by crates. The robot must verify if a cell is reachable before moving. A grid and `is_valid()` function signature are provided. Write the bounds and obstacle-check logic inside the function. Then take user input for row/col pairs and print the result for each.

```
# Q2
grid = [
    [0, 0, 1, 0, 0],
    [0, 1, 0, 0, 1],
    [0, 0, 0, 1, 0],
    [1, 0, 0, 0, 0],
    [0, 0, 1, 0, 0]
]

def is_valid(grid, row, col):
    # Check bounds and obstacles
    rows = len(grid)
    cols = len(grid[0])

    if row < 0 or row >= rows or col < 0 or col >= cols:
        return False

    return grid[row][col] == 0
```

```

try:
    num_cells = int(input('How many cells to check? '))

    for i in range(num_cells):
        print(f'--- Cell {i + 1} ---')

        r = int(input(' Enter row: '))
        c = int(input(' Enter col: '))

        print(f'is_valid({r}, {c}) = {is_valid(grid, r, c)}')

except ValueError:
    print('Please enter valid integers.')

```

```

How many cells to check? 3
--- Cell 1 ---
Enter row: 0
Enter col: 0
is_valid(0, 0) = True
--- Cell 2 ---
Enter row: 1
Enter col: 0
is_valid(1, 0) = True
--- Cell 3 ---
Enter row: 2
Enter col: 2
is_valid(2, 2) = True

```

Q3: Scenario: An explorer robot maps a cave system by diving as deep as possible into each tunnel before backtracking. A graph and empty DFS function signature are provided. Write the DFS logic using a stack: pop a node, mark it visited, print it, and push unvisited neighbors. Take a start node as input and print the visited order.

```

#Q3
graph = {
    'Cave1': ['Cave2', 'Cave3'],
    'Cave2': ['Cave1', 'Cave4', 'Cave5'],
    'Cave3': ['Cave1', 'Cave6'],
    'Cave4': ['Cave2'],
    'Cave5': ['Cave2', 'Cave7'],
    'Cave6': ['Cave3'],
    'Cave7': ['Cave5']
}

def dfs(graph, start):
    visited = []
    stack = [start]
    seen = set()

    while stack:
        node = stack.pop()

        if node not in seen:
            seen.add(node)
            visited.append(node)

            print(f'Visiting: {node}')

            for neighbor in reversed(graph[node]):
                if neighbor not in seen:
                    stack.append(neighbor)

    return visited

start_node = input('Enter start node: ').strip()

if start_node not in graph:
    print(f'Invalid node. Valid nodes: {list(graph.keys())}')
else:
    result = dfs(graph, start_node)
    print('Visited order:', result)

```

```

Enter start node: Cave1
Visiting: Cave1
Visiting: Cave2
Visiting: Cave4
Visiting: Cave5
Visiting: Cave7
Visiting: Cave3
Visiting: Cave6
Visited order: ['Cave1', 'Cave2', 'Cave4', 'Cave5', 'Cave7', 'Cave3', 'Cave6']

```

Q4: Scenario: A smart thermostat uses IF-THEN rules to automatically control home appliances based on sensor readings. A rules list, function signature, and input block are provided. Write the logic inside `fire_rules()`: loop through each rule, evaluate its condition, print triggered actions, and handle the no-trigger case.

```
#Q4
rules = [
    {
        'condition': lambda f: f.get('temperature', 0) > 35,
        'action': 'Turn ON AC'
    },
    {
        'condition': lambda f: f.get('temperature', 0) < 15,
        'action': 'Turn ON Heater'
    },
    {
        'condition': lambda f: f.get('smoke', False) == True,
        'action': 'FIRE ALERT - Sound Alarm'
    },
    {
        'condition': lambda f: f.get('motion', False) == True,
        'action': 'Turn ON Lights'
    }
]

def fire_rules(facts):
    # Evaluate all rules
    triggered = []

    for rule in rules:
        if rule['condition'](facts):
            triggered.append(rule['action'])
            print(f'Action triggered: {rule["action"]}')

    if not triggered:
        print('No rules triggered.')

    return triggered

try:
    temp = int(input('Enter temperature: '))

    smoke = input('Smoke detected? (yes/no): ').strip().lower() == 'yes'
    motion = input('Motion detected? (yes/no): ').strip().lower() == 'yes'

    facts = {
        'temperature': temp,
        'smoke': smoke,
        'motion': motion
    }

    print('--- Evaluating Rules ---')
    fire_rules(facts)

except ValueError:
    print('Invalid input. Enter a valid integer for temperature.')
```

```
Enter temperature: 40
Smoke detected? (yes/no): no
Motion detected? (yes/no): no
--- Evaluating Rules ---
Action triggered: Turn ON AC
```

Q5: Scenario: A traveler wants to view all road options from their current city, sorted from cheapest to most expensive toll cost. A weighted graph and empty function signature are provided. Write the logic inside `print_sorted_neighbors()` to sort neighbors by cost and print them. Then take a node name as input.

```
#Q5
graph = {
    'A': {'B': 4, 'C': 2, 'D': 7},
    'B': {'A': 4, 'C': 1, 'E': 5},
    'C': {'A': 2, 'B': 1, 'D': 3, 'E': 6},
    'D': {'A': 7, 'C': 3},
    'E': {'B': 5, 'C': 6}
}

def print_sorted_neighbors(graph, node):
    # Sort neighbors by edge cost
    neighbors = graph[node]
```

```

sorted_neighbors = sorted(neighbors.items(), key=lambda x: x[1])

print(f'From {node} (sorted by cost):')

for neighbor, cost in sorted_neighbors:
    print(f' -> {neighbor}: cost {cost}')

node = input('Enter node: ').strip()

if node in graph:
    print_sorted_neighbors(graph, node)
else:
    print('Node not found.')

```

```

Enter node: A
From A (sorted by cost):
-> C: cost 2
-> B: cost 4
-> D: cost 7

```

Q6: Scenario: A security robot patrols a 4×4 office grid. It must visit every cell and immediately report any alert zones it detects. The import, function signature, alert_count init, outer for-loop line, and input/grid-generation block are provided. Write the inner loop: visit each cell in each row, print the correct status message, and increment the alert counter

```

#Q6
import random

def patrol_agent(grid):
    # Patrol the grid and detect alerts
    alert_count = 0

    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 'alert':
                print(f'ALERT at ({r},{c})!')
                alert_count += 1
            else:
                print(f'Patrolling ({r},{c})')

    print(f'Total alerts found: {alert_count}')

try:
    seed = int(input('Enter random seed: '))

    random.seed(seed)

    grid = [
        ['alert' if random.random() < 0.3 else 'safe' for _ in range(4)]
        for _ in range(4)
    ]

    print('Grid layout:')
    for row in grid:
        print(row)

    patrol_agent(grid)

except ValueError:
    print('Please enter a valid integer for the seed.')

```

```

Enter random seed: 2
Grid layout:
['safe', 'safe', 'alert', 'alert']
['safe', 'safe', 'safe', 'safe']
['safe', 'safe', 'safe', 'alert']
['safe', 'safe', 'safe', 'safe']
Patrolling (0,0)
Patrolling (0,1)
ALERT at (0,2)!
ALERT at (0,3)!
Patrolling (1,0)
Patrolling (1,1)
Patrolling (1,2)
Patrolling (1,3)
Patrolling (2,0)
Patrolling (2,1)
Patrolling (2,2)
ALERT at (2,3)!
Patrolling (3,0)
Patrolling (3,1)

```

```
Patrolling (3,2)
Patrolling (3,3)
Total alerts found: 3
```

Q7: Scenario: A rescue robot estimates how far it needs to travel to reach a trapped survivor using only grid coordinates. An empty function signature for `manhattan_distance()` is provided. Write the formula inside. Then take user input for how many pairs to test, read each pair, and print the result.

```
#Q7
def manhattan_distance(current, goal):
    # Calculate Manhattan Distance
    return abs(current[0] - goal[0]) + abs(current[1] - goal[1])

try:
    n = int(input('How many pairs to test? '))

    for i in range(n):
        print(f'--- Pair {i + 1} ---')

        r1 = int(input(' Current row: '))
        c1 = int(input(' Current col: '))

        r2 = int(input(' Goal row: '))
        c2 = int(input(' Goal col: '))

        d = manhattan_distance((r1, c1), (r2, c2))

        print(f'From ({r1},{c1}) to ({r2},{c2}) -> distance: {d}')

except ValueError:
    print('Please enter valid integers.')
```

```
How many pairs to test? 2
--- Pair 1 ---
Current row: 1
Current col: 2
Goal row: 4
Goal col: 6
From (1,2) to (4,6) -> distance: 7
--- Pair 2 ---
Current row: 0
Current col: 0
Goal row: 3
Goal col: 5
From (0,0) to (3,5) -> distance: 8
```

Q8: Scenario: A GPS system stores fuel costs between road intersections and must recommend the cheapest next turn from any location. A weighted graph and empty `cheapest_neighbor()` signature are provided. Write the logic to find the minimum-cost neighbor using `min()`. Then take a node as input and print the result.

```
#Q8
graph = {
    'A': {'B': 4, 'C': 2, 'D': 7},
    'B': {'A': 4, 'C': 1, 'E': 5},
    'C': {'A': 2, 'B': 1, 'D': 3, 'E': 6},
    'D': {'A': 7, 'C': 3},
    'E': {'B': 5, 'C': 6}
}

def cheapest_neighbor(graph, node):
    # Student writes min-cost neighbor logic here
    neighbors = graph[node]
    cheapest = min(neighbors, key=lambda x: neighbors[x])
    return cheapest, neighbors[cheapest]

node = input('Enter node: ')

if node in graph:
    neighbor, cost = cheapest_neighbor(graph, node)
    print(f'Cheapest neighbor of {node}: {neighbor} with cost {cost}')
else:
    print('Node not found.')
```

```
Enter node: B
Cheapest neighbor of B: C with cost 1
```

Q9: Scenario: A warehouse robot needs to store its current shelf coordinates and its delivery target before it starts moving. No starter code is given. Take current row, current col, goal row, and goal col as user input. Store all four in a list called `route_info`. Print current

position, goal position, and the full list using f-strings.

```
# Q9
try:
    curr_row = int(input('Enter current row: '))
    curr_col = int(input('Enter current col: '))
    goal_row = int(input('Enter goal row: '))
    goal_col = int(input('Enter goal col: '))

    route_info = [curr_row, curr_col, goal_row, goal_col]

    print(f'Current Position -> Row: {route_info[0]}, Col: {route_info[1]}')
    print(f'Goal Position -> Row: {route_info[2]}, Col: {route_info[3]}')
    print(f'route_info: {route_info}')

except ValueError:
    print('Please enter valid integers.')
```

```
Enter current row: 0
Enter current col: 0
Enter goal row: 2
Enter goal col: 2
Current Position -> Row: 0, Col: 0
Goal Position -> Row: 2, Col: 2
route_info: [0, 0, 2, 2]
```

Q10: Scenario: A fire-fighting robot must reach a burning room in the fewest possible steps through a grid-based building. The grid, function signature, data initialisation, loop skeleton, and input block are provided. Inside the while loop, write: the goal check and return, and the 4-direction neighbour expansion with bounds/obstacle/visited checks.

```
# Q10
from collections import deque

grid = [
    [0,0,0,0,0,0],
    [0,1,1,0,1,0],
    [0,0,0,0,1,0],
    [0,1,0,0,0,0],
    [0,1,0,1,1,0],
    [0,0,0,0,1,0]
]

def bfs_grid(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    queue = deque([(start, [start])])
    visited = set([start])
    nodes_explored = 0

    while queue:
        (r, c), path = queue.popleft()
        nodes_explored += 1

        # Student: check if goal reached, return path and count
        if (r, c) == goal:
            return path, nodes_explored

        # Student: expand 4 neighbours, check bounds/obstacle/visited, append to queue
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nr, nc = r + dr, c + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0 and (nr, nc) not in visited:
                visited.add((nr, nc))
                queue.append(((nr, nc), path + [(nr, nc)]))

    return None, nodes_explored

try:
    sr = int(input('Start row: '))
    sc = int(input('Start col: '))
    gr = int(input('Goal row: '))
    gc = int(input('Goal col: '))

    ROWS, COLS = len(grid), len(grid[0])

    if not (0 <= sr < ROWS and 0 <= sc < COLS and 0 <= gr < ROWS and 0 <= gc < COLS):
        print(f'Coordinates out of range. Grid is {ROWS}x{COLS} (0-indexed).')

    elif grid[sr][sc] == 1 or grid[gr][gc] == 1:
        print('Start or goal is an obstacle. Choose a free cell (value 0).')
```

```

else:
    path, explored = bfs_grid(grid, (sr, sc), (gr, gc))

    if path:
        print('Shortest path:', path)
        print('Steps:', len(path) - 1)
        print('Nodes explored:', explored)
    else:
        print('No path found. Nodes explored:', explored)

except ValueError:
    print('Please enter valid integers.')

```

```

Start row: 1
Start col: 2
Goal row: 4
Goal col: 3
Start or goal is an obstacle. Choose a free cell (value 0).

```

Q11: Scenario: A city navigation app needs to display all road connections between cities without printing any edge twice. A city graph is provided. Write a function `print_edges(graph)` that uses a seen set to avoid duplicates and prints each edge as `CityA -- CityB`. Then take a city name as input and print only that city's direct connections.

```

# Q11
graph = {
    'Hyderabad': ['Mumbai', 'Chennai', 'Bangalore'],
    'Mumbai': ['Hyderabad', 'Delhi', 'Pune'],
    'Chennai': ['Hyderabad', 'Bangalore'],
    'Bangalore': ['Hyderabad', 'Chennai', 'Pune'],
    'Delhi': ['Mumbai'],
    'Pune': ['Mumbai', 'Bangalore']
}

def print_edges(graph):
    # Student writes dedup-edge print logic here
    seen = set()

    for city in graph:
        for neighbor in graph[city]:
            edge = tuple(sorted([city, neighbor]))

            if edge not in seen:
                seen.add(edge)
                print(f'{city} -- {neighbor}')

print('All road connections:')
print_edges(graph)

city = input('Enter a city to see its connections: ').strip()

if city in graph:
    print(f'Connections of {city}:')
    for neighbor in graph[city]:
        print(f' {city} -- {neighbor}')
else:
    print('City not found.')

```

```

All road connections:
Hyderabad -- Mumbai
Hyderabad -- Chennai
Hyderabad -- Bangalore
Mumbai -- Delhi
Mumbai -- Pune
Chennai -- Bangalore
Bangalore -- Pune
Enter a city to see its connections: HYD
City not found.

```

Q12: Scenario: A maze-solving robot must find any path from entrance to exit of a grid dungeon by exploring as deeply as possible before backtracking. The grid, function signature, data init, stack loop skeleton, and input block are provided. Inside the while loop, write: mark node visited, increment counter, check if goal, expand 4 neighbours with checks, and push to stack.

```

# Q12
grid = [
    [0,0,0,0,0,0],
    [0,1,1,0,1,0],
    [0,0,0,0,1,0],
    [0,1,0,0,0,0],
    [0,1,0,1,1,0],
    [0,0,0,0,0,0]
]

```

```

[0,0,0,0,1,0]
]

def dfs_grid(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    stack = [(start, [start])]
    visited = set()
    nodes_visited = 0

    while stack:
        (r, c), path = stack.pop()

        if (r, c) in visited:
            continue

        # Student: mark visited, increment counter
        visited.add((r, c))
        nodes_visited += 1

        # Student: check goal, expand neighbours, push to stack
        if (r, c) == goal:
            return path, nodes_visited

        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            nr, nc = r + dr, c + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0 and (nr, nc) not in visited:
                stack.append(((nr, nc), path + [(nr, nc)]))

    return None, nodes_visited

try:
    sr = int(input('Start row: '))
    sc = int(input('Start col: '))
    gr = int(input('Goal row: '))
    gc = int(input('Goal col: '))

    ROWS, COLS = len(grid), len(grid[0])

    if not (0 <= sr < ROWS and 0 <= sc < COLS and 0 <= gr < ROWS and 0 <= gc < COLS):
        print(f'Coordinates out of range. Grid is {ROWS}x{COLS} (0-indexed).')

    elif grid[sr][sc] == 1 or grid[gr][gc] == 1:
        print('Start or goal is an obstacle. Choose a free cell (value 0).')

    else:
        path, count = dfs_grid(grid, (sr, sc), (gr, gc))

        if path:
            print('Path found:', path)
            print('Nodes visited:', count)
        else:
            print('No path found. Nodes visited:', count)

except ValueError:
    print('Please enter valid integers.')

```

```

Start row: 0
Start col: 0
Goal row: 2
Goal col: 2
Path found: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 5), (2, 5), (3, 5), (3, 4), (3, 3), (3, 2), (2, 2)]
Nodes visited: 13

```

Q13: Scenario: A cleaning robot is deployed in a hallway of 6 rooms. It moves left to right and cleans any dirty room it finds. The import, function signature, cleaned counter, and input/grid block are provided. Write the for-loop body inside `reflex_agent()`: check each room's state, print the correct message, and increment the counter.

```

# Q13
import random

def reflex_agent(rooms):
    # Student writes loop + if-else logic here
    cleaned = 0

    for i, state in enumerate(rooms):
        if state == 'dirty':
            print(f'Cleaning room {i}')
            cleaned += 1
        else:
            print(f'Room {i} already clean')

```

```

    print(f'Total rooms cleaned: {cleaned}')

try:
    seed = int(input('Enter random seed: '))
    random.seed(seed)

    rooms = [random.choice(['dirty', 'clean']) for _ in range(6)]

    print('Initial room states:', rooms)
    reflex_agent(rooms)

except ValueError:
    print('Please enter a valid integer for the seed.')

```

```

Enter random seed: 4
Initial room states: ['dirty', 'clean', 'dirty', 'clean', 'clean', 'dirty']
Cleaning room 0
Room 1 already clean
Cleaning room 2
Room 3 already clean
Room 4 already clean
Cleaning room 5
Total rooms cleaned: 3

```

Q14: Scenario: A logistics robot must find the minimum fuel-cost route to deliver a package across a city where every road has a different cost. An import, weighted graph, function signature, pq/visited init, and the while-loop + heappop line are provided. Write the loop body: skip visited nodes, mark as visited, check goal, expand neighbours with heappush.

```

# Q14
import heapq

graph = {
    'A': [('B',1),('C',4)],
    'B': [('A',1),('C',2),('D',5)],
    'C': [('A',4),('B',2),('D',1),('E',3)],
    'D': [('B',5),('C',1),('F',2)],
    'E': [('C',3),('F',1),('G',4)],
    'F': [('D',2),('E',1),('G',2)],
    'G': [('E',4),('F',2)]
}

def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return path, cost

        for neighbor, edge_cost in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq, (cost + edge_cost, neighbor, path + [neighbor]))

    return None, float('inf')

start = input('Start node: ').strip()
goal = input('Goal node: ').strip()

if start not in graph or goal not in graph:
    print(f'Invalid node. Valid nodes: {list(graph.keys())}')
else:
    path, total_cost = ucs(graph, start, goal)

    if path:
        print('Path:', ' -> '.join(path))
        print('Total cost:', total_cost)
    else:
        print('No path found')

```

```

Start node: B
Goal node: D
Path: B -> C -> D
Total cost: 3

```

Q15: Scenario: A delivery drone logs each rooftop it hops across and the fuel cost of each hop to report total usage. A costs dictionary, function signature, and total=0 init are provided. Write the accumulation loop inside total_path_cost(): add each node's cost, print the running total per step. Then take a space-separated path as input

```
# Q15
costs = {'A': 2, 'B': 3, 'C': 1, 'D': 4, 'E': 2}

def total_path_cost(path, costs):
    # Student writes accumulation loop here
    total = 0

    for node in path:
        total += costs[node]
        print(f'Node {node} -> cost {costs[node]} -> running total: {total}')

    return total

try:
    raw = input('Enter path nodes separated by spaces: ').strip().upper()
    path = raw.split()

    invalid = [n for n in path if n not in costs]

    if invalid:
        print(f'Unknown nodes: {invalid}. Valid: {list(costs.keys())}')
    else:
        result = total_path_cost(path, costs)
        print(f'Final total cost: {result}')

except Exception as e:
    print(f'Error: {e}')
```

```
Enter path nodes separated by spaces: A D
Node A -> cost 2 -> running total: 2
Node D -> cost 4 -> running total: 6
Final total cost: 6
```

Q16: Scenario: A robot in a smart warehouse must find the optimal path to a package by combining actual travel cost and estimated remaining distance. The import, grid, heuristic function, astar() signature, all initialisation, the while loop skeleton with visited-check/mark/counter/goal-return are provided. Write only the neighbour expansion: loop 4 directions, check bounds/obstacles/visited, compute ng=g+1 and nf=ng+heuristic, heappush to pq.

```
#Q16
import heapq

grid = [
    [0,0,0,0,0,0],
    [0,1,1,0,1,0],
    [0,0,0,0,1,0],
    [0,1,0,0,0,0],
    [0,1,0,1,1,0],
    [0,0,0,0,1,0]
]

def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    pq = [(heuristic(start,goal), 0, start, [start])]
    visited = set()
    nodes_expanded = 0

    while pq:
        f, g, (r,c), path = heapq.heappop(pq)

        if (r,c) in visited:
            continue

        visited.add((r,c))
        nodes_expanded += 1

        if (r,c) == goal:
            return path, g, nodes_expanded

        # Student: loop 4 directions, check bounds+obstacle+visited
        # compute ng=g+1, nf=ng+heuristic, heappush to pq
        for dr,dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr,nc = r+dr, c+dc
```

```

        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==0 and (nr,nc) not in visited:
            ng = g + 1
            heapq.heappush(
                pq,
                (ng + heuristic((nr,nc),goal), ng, (nr,nc), path + [(nr,nc)]))
            )

    return None, float('inf'), nodes_expanded

try:
    sr = int(input('Start row: '))
    sc = int(input('Start col: '))
    gr = int(input('Goal row: '))
    gc = int(input('Goal col: '))

    ROWS, COLS = len(grid), len(grid[0])

    if not (0<=sr<ROWS and 0<=sc<COLS and 0<=gr<ROWS and 0<=gc<COLS):
        print(f'Coordinates out of range. Grid is {ROWS}x{COLS} (0-indexed).')

    elif grid[sr][sc] == 1 or grid[gr][gc] == 1:
        print('Start or goal is an obstacle. Choose a free cell (value 0).')

    else:
        path, cost, expanded = astar(grid, (sr,sc), (gr,gc))

        if path:
            print('Optimal path:', path)
            print('Total cost:', cost)
            print('Nodes expanded:', expanded)
        else:
            print('No path found')

except ValueError:
    print('Please enter valid integers.')

```

```

Start row: 1
Start col: 1
Goal row: 2
Goal col: 2
Start or goal is an obstacle. Choose a free cell (value 0).

```